

SQL Star Schema, Load & KPI Layer

SQL Server built from scratch, 46,601 clean rows loaded and reconciled, three certified KPI views verified against real numbers.

Prepared by	Neelam Lahre		
Platform	SQL Server 2022 Express, database prudential_dw		
Scripts	sql/01_schema.sql, sql/02_views.sql, python/03_load.py, python/03b_load_claims.py		
Guide reference	Solutioning Guide v1.0, Stages 7, 8 and 9		
Status	Stages 1-9 complete. Stage 10 (simulation engine) not yet started.		

11	46,188	27,583	3 / 3
tables built	policy rows loaded	claim rows loaded	KPI views verified

Stage 7 — Build the SQL Star Schema

Goal: dimensions describe, facts measure — one governed shape.

SQL Server 2022 Express and SSMS were installed from scratch for this stage (no prior SQL Server instance existed on the machine). Connectivity required two fixes typical of a fresh local install: the ODBC Driver 17 for SQL Server package, installed separately from SQL Server itself, and TrustServerCertificate=yes to accept the instance's self-signed certificate.

Postgres → SQL Server syntax adapted

Guide (Postgres)	This build (T-SQL)	Why
SERIAL / BIGSERIAL	INT/BIGINT IDENTITY(1,1)	No native auto-increment keyword
BOOLEAN	BIT	No native boolean type
GENERATED ALWAYS AS ... STORED	AS (...) PERSISTED	Different computed-column syntax
NOW()	SYSDATETIME()	Different current-timestamp function
FILTER (WHERE ...)	SUM(CASE WHEN ... THEN x ELSE 0 END)	No FILTER clause in T-SQL
STDDEV	STDEV	One fewer letter
CREATE OR REPLACE VIEW	CREATE OR ALTER VIEW	Different keyword
PERCENTILE_CONT ... GROUP BY	PERCENTILE_CONT ... OVER(PARTITION BY ...)	Requires a window, not plain GROUP BY

Schema extended beyond the guide's excerpt: the guide shows only two example dimensions (dim_product, dim_agent) and implies the rest by pattern. This build adds dim_date, dim_customer, dim_channel, dim_assessor and dim_claim_type explicitly, plus fact_claim in full — none of these are spelled out in the source document.

Proved: all 11 tables confirmed present in prudential_dw (7 dimensions, 2 facts, 2 scenario store tables) via INFORMATION_SCHEMA.TABLES.

Stage 8 — Load the Clean Data into SQL

Goal: dimensions first, then facts — order matters.

Six real defects surfaced during this stage, each caught and fixed before data reached the warehouse rather than silently corrupting it:

#	Issue found	Fix
1	pyodbc not installed in the active venv	Installed inside the activated .venv
2	dim_date's date column returned as Python date objects, not datetime64 -- merge failed	pd.to_datetime() cast after read_sql
3	46 rows: product_category present, product_name missing -- no dim_product match	Quarantined, reason NO_PRODUCT_DIM_MATCH
4	persistency_13m_flag (source) vs persistency_13m (warehouse) name mismatch	Renamed before column selection
5	months_inforce, care_delay_days never cast to numeric in Stages 3-6	pd.to_numeric() added at load time
6	367 rows: annual_premium_sgd <= 0, violates warehouse CHECK constraint	Quarantined, reason NON_POSITIVE_PREMIUM

Issue #6's 367 rows are the same 367 the DQ report already flagged as annual_premium_sgd_out_of_range -- and the same count age_out_of_range showed *before* the age fix landed. Confirms these are deliberately-seeded rows carrying more than one defect at once, now caught at two separate points in the pipeline rather than reaching the warehouse.

Reconciliation: policies

Stage	Rows
Clean rows entering Stage 8	46,601
Quarantined -- no product_name match	-46
Quarantined -- non-positive premium	-367
Loaded into fact_policy_new_business	46,188

Reconciliation: claims (fact_claim)

Stage	Rows
Rows with a claim_id	27,818
Duplicate claim_id found in source (checked, none in final set)	0
Quarantined -- parent policy not loaded at Stage 8	-235
Loaded into fact_claim	27,583

dim_claim_type also had a gap the guide never addressed: claim_type was never standardised in Stage 6.1 (only 4 of the extract's categorical fields were). A trim + title-case normalise was added at load time to stop casing variants fragmenting into duplicate categories -- 8 clean types loaded.

Proved: SELECT COUNT(*) FROM fact_policy_new_business returns 46,188 · SELECT COUNT(*) FROM fact_claim returns 27,583 · both quarantine files exist with a machine-readable reject_reason.

Stage 9 — The Business KPI Layer

Goal: every dashboard reads these views -- no local reinterpretation.

All three views (vw_nbp_margin for BO-1, vw_health_gap for BO-2, vw_assessor_alerts for BO-5) built and executed without syntax errors -- but the first test of vw_nbp_margin returned **every row dated 1899-01**, sorted to the top by year_month. That was the surface symptom of a real gap, not a rendering glitch.

Root cause and fix

Stage 5 flags impossible date relationships (policy_start_date before 1990, after today, a quote after its own policy start, etc.) into a dq_flag_date column -- but Stage 6.3's dq_row_status is built *only* from the RULES dictionary, which never included policy_start_date. A row with a genuinely corrupt policy_start_date (~1899) could carry dq_row_status = 'OK' straight into the warehouse, and vw_nbp_margin's WHERE f.dq_row_status = 'OK' filter let it through -- surfacing at the top of an ORDER BY year_month list purely because "1899" sorts before "20XX" alphabetically. This is also the same outlier flagged earlier as the cause of dim_date's oversized 73,414-row span.

Fix: folded Stage 5's date-logic violations into the same violations flag that drives dq_row_status, so a bad date is treated exactly like a bad age or premium -- flagged REVIEW, excluded from certified KPI views, never silently trusted.

dq_row_status	Before fix	After fix
OK	45,784	45,658
REVIEW	817	943

Verification after fix and full reload

View	Result
vw_nbp_margin	Real dates (2023-01 onward), blended_margin 0.10-0.29, low_margin_mix correctly 0/1 by product
vw_health_gap	Loss ratio rises monotonically with delay: 0.261 -> 0.269 -> 0.275 -> 0.306 -> 0.360 -- the Health Gap, proven in this data
vw_assessor_alerts	3 assessors separate clearly at z-score ~4.3, versus the rest clustered near 0 -- the fraud signal BO-5 requires

Proved: all three views return rows consistent with the business hypotheses they were built to test, using only certified fields (dq_row_status = 'OK') -- the single-version-of-truth objective (BO-7) is now backed by a warehouse that actually enforces it.

Day 3 (Stages 7-9) complete. Next: Stage 10 -- the Python Monte Carlo simulation engine (S-01 to S-08), reading its baseline directly from this warehouse.